Project Title:

Arduino MEGA Multi-Sensor Monitoring and Control System

Author Name:

Heta Patel

DO NOT COPY

2. Table of Contents

I. Abstract

II. Introduction

III. Materials and Components

IV. System Design

V. Working and Operation

VI. Code Summary

VII. Testing and Results

VIII. Challenges Faced

IX. Conclusion

X. References

XI. Appendices

DO NOT COPY

I. Abstract

In this project, we built a multi-sensor monitoring and control system using two Arduino MEGA boards. The idea was to connect different sensors and make them work together in a smart way.

The system uses components like a keypad, LCD, DHT11 sensor, PIR sensor, ultrasonic sensor, and LDR. One Arduino (MEGA-1) handles user input like passcode and temperature, while the other (MEGA-2) handles sensors and controls outputs like LEDs, buzzer, and motor.

Overall, the system was able to detect motion, measure temperature, respond to light conditions, and control a motor based on conditions. It gave a good understanding of how real-time systems work.

II. Introduction

Arduino is widely used for building small electronics projects where different components need to work together. Interfacing is basically connecting sensors and devices to the Arduino and making them communicate properly.

In this project, we tried to build something similar to a **smart room system**. It can monitor the environment and take actions automatically.

The motivation behind this project was to:

- Learn how to connect multiple sensors
- Understand real-time decision making
- Use communication between two Arduino boards (I2C)

This project also relates to real-life applications like smart homes and safety systems.

DO NOT COPY

III. Materials and Components

Component	Quantity	Description
Arduino MEGA	2	Main controllers
LCD 16x2	2	Display messages
Keypad	1	Input for passcode and temperature
DHT11	1	Temperature and humidity sensor
PIR Sensor	1	Detects motion
Ultrasonic Sensor	1	Measures distance
LDR	1	Detects light
LEDs	8	Indicators
Buzzer	2	Alerts
74HC595	1	Shift register
7-segment display	1	Displays passcode
Motor	1	Fan simulation
Resistors	Many	Circuit safety
Potentiometer	2	LCD contrast

DO NOT COPY

IV. System Design

Block Diagram (Text)

User → Keypad → MEGA-1 → I2C → MEGA-2 → Sensors → Output

Pin Mapping (Important)

MEGA-1

Component	Pins
LCD	12,11,5,4,3,2
Keypad	22–29
Push Button	30
Buzzer	31
74HC595	32,33,34
Digits	35–38

MEGA-2

Component	Pins
LCD	22–27
DHT11	28
PIR	29
Ultrasonic	30,31
Buzzer	43
Yellow LEDs	48–51
Red LEDs	44–47
LDR	A0
Motor	2

I2C Connection

MEGA-1	MEGA-2
20	20
21	21
GND	GND

This part was very important. At first, the boards were not communicating, and later we realized GND must be common, otherwise nothing works.

The system architecture is shown in Appendix B

DO NOT COPY

V. Working and Operation

Step 1: Passcode

User enters a 4-digit passcode using keypad.
If it matches, system continues. If not, it restarts.

Step 2: Temperature Input

User enters a value like 235 → system converts it to 23.5°C.
It also shows the value in Fahrenheit.

Step 3: Start System

User presses push button (PB1).
System shows **"System is ON"** and sends data to MEGA-2.

Step 4: Sensor Operation

- PIR detects motion → yellow LEDs turn ON
- DHT reads temperature
- Ultrasonic checks distance
- LDR checks light

Step 5: Control Logic

- If temperature is high → motor ON
- If object is too close → motor OFF + buzzer
- If dark → red LEDs ON

The working flow of the system is illustrated in Appendix B

DO NOT COPY

VI. Code Summary

The code is divided into two parts.

MEGA-1 Code

- Handles keypad input
- Verifies passcode
- Displays messages
- Sends temperature using I2C

MEGA-2 Code

- Reads sensors
- Controls LEDs, buzzer, and motor
- Applies conditions based on inputs

Important Logic

- Sensor reading happens continuously
- Display updates based on inputs
- Motor control depends on temperature + distance
- Button is used to start system

DO NOT COPY

VII. Testing and Results

Testing

We tested each component one by one (very important).

Feature	Result
Passcode	Worked
LCD	Worked
Temperature input	Accurate
PIR	Worked
Ultrasonic	Worked
LDR	Worked
Motor	Worked

Observations

- System responded properly most of the time
- Sensors worked as expected
- LCD display was clear

Issues Faced During Testing

- DHT sensor sometimes showed error
- Initially I2C was not working
- Wiring mistakes caused delays
- Motor didn't work at first because of wrong connection

DO NOT COPY

VIII. Challenges Faced

Some challenges we faced:

- Too many wires → confusing connections
- Debugging I2C communication
- Sensor readings not stable sometimes
- Timing between sensors
- Understanding motor circuit (transistor + diode)

IX. Conclusion

In this project, we learned how to connect multiple sensors and control them using Arduino. It helped us understand real-time systems and hardware interfacing better.

We also learned how important proper wiring and step-by-step testing is.

If we improve this project in future, we can:

- Add mobile control
- Connect it to internet (IoT)
- Improve accuracy of sensors

X. References

- Arduino official website
- Sensor datasheets
- Online tutorials

DO NOT COPY

XI. Appendices

Appendix A

MEGA-1 code

```
#include <Keypad.h>
#include <LiquidCrystal.h>
#include <Wire.h>

const byte mega2Address = 8;

// LCD
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

// Keypad
const byte ROWS = 4;
const byte COLS = 4;

char keys[ROWS][COLS] = {
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','0','#','D'}
};

byte rowPins[ROWS] = {29, 28, 27, 26};
byte colPins[COLS] = {25, 24, 23, 22};

Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

// Buzzer
const int buzzerPin = 31;

// PB1
const int pb1Pin = 30;

// 74HC595 + 4-digit 7-segment
const int dataPin = 32;
const int clockPin = 33;
const int latchPin = 34;
const int digitPins[4] = {35, 36, 37, 38};

// Common cathode codes
byte digits[10] = {
  B00111111,
  B00000110,
  B01011011,
```

```
  B01001111,
  B01100110,
  B01101101,
  B01111101,
  B00000111,
  B01111111,
  B01101111
};

// Stages
// 1 = first pass
// 2 = second pass
// 3 = temp input
// 4 = wait for PB1
// 5 = system on
int stage = 1;

String pass1 = "";
String pass2 = "";
String tempInput = "";

float preferredTempC = 0.0;
float preferredTempF = 0.0;

void setup() {
  Wire.begin();

  lcd.begin(16, 2);
  lcd.clear();
  lcd.print("Enter Pass:");
  lcd.setCursor(0, 1);
  lcd.print("____");

  pinMode(buzzerPin, OUTPUT);
  digitalWrite(buzzerPin, LOW);

  // use internal pull-up
  pinMode(pb1Pin, INPUT_PULLUP);

  pinMode(dataPin, OUTPUT);
  pinMode(clockPin, OUTPUT);
  pinMode(latchPin, OUTPUT);

  for (int i = 0; i < 4; i++) {
    pinMode(digitPins[i], OUTPUT);
    digitalWrite(digitPins[i], HIGH);
  }

  clearDisplay();
}

void loop() {
  if (stage == 4) {
```

```
    waitForPB1();
    return;
  }

  char key = keypad.getKey();

  if (key) {
    if (stage == 1) {
      handleFirstPass(key);
    }
    else if (stage == 2) {
      handleSecondPass(key);
    }
    else if (stage == 3) {
      handleTempInput(key);
    }
  }
}

void handleFirstPass(char key) {
  if (key == '*') {
    pass1 = "";
    lcd.clear();
    lcd.print("Enter Pass:");
    lcd.setCursor(0, 1);
    lcd.print("____");
  }
  else if (key == '#') {
    if (pass1.length() == 4) {
      stage = 2;
      lcd.clear();
      lcd.print("Re-enter Pass:");
      lcd.setCursor(0, 1);
      lcd.print("____");
    }
  }
  else if (key >= '0' && key <= '9') {
    if (pass1.length() < 4) {
      pass1 += key;
      lcd.setCursor(0, 1);
      lcd.print("____");
      lcd.setCursor(0, 1);
      lcd.print(pass1);
    }
  }
}

void handleSecondPass(char key) {
  if (key == '*') {
    pass2 = "";
    lcd.clear();
    lcd.print("Re-enter Pass:");
    lcd.setCursor(0, 1);
```

```
    lcd.print("____");
  }
  else if (key == '#') {
    if (pass2.length() == 4) {
      if (pass1 == pass2) {
        lcd.clear();
        lcd.print("Matched!");
        delay(1000);

        digitalWrite(buzzerPin, HIGH);
        delay(1000);
        digitalWrite(buzzerPin, LOW);

        flashDisplay(pass1.toInt());
        clearDisplay();

        stage = 3;
        tempInput = "";

        lcd.clear();
        lcd.print("Set Temp:");
        lcd.setCursor(0, 1);
        lcd.print("___");
      }
      else {
        lcd.clear();
        lcd.print("Mismatch!");
        delay(2000);

        pass1 = "";
        pass2 = "";
        stage = 1;

        lcd.clear();
        lcd.print("Enter Pass:");
        lcd.setCursor(0, 1);
        lcd.print("____");
      }
    }
  }
  else if (key >= '0' && key <= '9') {
    if (pass2.length() < 4) {
      pass2 += key;
      lcd.setCursor(0, 1);
      lcd.print("____");
      lcd.setCursor(0, 1);
      lcd.print(pass2);
    }
  }
}

void handleTempInput(char key) {
  if (key == '*') {
```

```
    tempInput = "";
    lcd.clear();
    lcd.print("Set Temp:");
    lcd.setCursor(0, 1);
    lcd.print("___");
  }
  else if (key == '#') {
    if (tempInput.length() == 3) {
      int rawValue = tempInput.toInt();
      preferredTempC = rawValue / 10.0;
      preferredTempF = (preferredTempC * 9.0 / 5.0) + 32.0;

      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print(preferredTempC, 1);
      lcd.print((char)223);
      lcd.print("C");

      lcd.setCursor(0, 1);
      lcd.print(preferredTempF, 1);
      lcd.print((char)223);
      lcd.print("F");

      delay(3000);

      stage = 4;
      lcd.clear();
      lcd.print("Press PB1");
      lcd.setCursor(0, 1);
      lcd.print("to Start");
    }
  }
  else if (key >= '0' && key <= '9') {
    if (tempInput.length() < 3) {
      tempInput += key;

      lcd.setCursor(0, 1);
      lcd.print("                ");
      lcd.setCursor(0, 1);

      if (tempInput.length() == 1) {
        lcd.print(tempInput[0]);
      }
      else if (tempInput.length() == 2) {
        lcd.print(tempInput[0]);
        lcd.print(tempInput[1]);
      }
      else if (tempInput.length() == 3) {
        lcd.print(tempInput[0]);
        lcd.print(tempInput[1]);
        lcd.print(".");
        lcd.print(tempInput[2]);
        lcd.print((char)223);
```

```
        lcd.print("C");
      }
    }
  }
}

void waitForPB1() {
  Serial.begin(9600);
  if (digitalRead(pb1Pin) == LOW) {
    lcd.clear();
    lcd.print("System is ON");

    int tempToSend = tempInput.toInt();   // example: 300 = 30.0C

    Wire.beginTransmission(mega2Address);
    Wire.write(highByte(tempToSend));
    Wire.write(lowByte(tempToSend));
    Wire.endTransmission();

    stage = 5;
  }
}

void flashDisplay(int num) {
  unsigned long startTime = millis();

  while (millis() - startTime < 5000) {
    unsigned long onStart = millis();
    while (millis() - onStart < 500) {
      displayNumber(num);
    }

    clearDisplay();
    delay(500);
  }
}

void displayNumber(int num) {
  int thousands = num / 1000;
  int hundreds = (num / 100) % 10;
  int tens = (num / 10) % 10;
  int ones = num % 10;

  showDigit(0, thousands);
  showDigit(1, hundreds);
  showDigit(2, tens);
  showDigit(3, ones);
}

void showDigit(int digitIndex, int value) {
  for (int i = 0; i < 4; i++) {
    digitalWrite(digitPins[i], HIGH);
  }
```

```
  digitalWrite(latchPin, LOW);
  shiftOut(dataPin, clockPin, MSBFIRST, digits[value]);
  digitalWrite(latchPin, HIGH);

  digitalWrite(digitPins[digitIndex], LOW);
  delay(3);
  digitalWrite(digitPins[digitIndex], HIGH);
}

void clearDisplay() {
  for (int i = 0; i < 4; i++) {
    digitalWrite(digitPins[i], HIGH);
  }

  digitalWrite(latchPin, LOW);
  shiftOut(dataPin, clockPin, MSBFIRST, 0);
  digitalWrite(latchPin, HIGH);
}
```

MEGA-2 code

```
#include <Wire.h>
#include <LiquidCrystal.h>
#include <DHT.h>
#include <SevSeg.h>

LiquidCrystal lcd(22, 23, 24, 25, 26, 27);

#define DHTPIN 28

#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

SevSeg sevseg;

// I2C

int receivedTempRaw = 235;

float preferredTemp = 23.5;

float activePreferredTemp = 23.5;

// Siren

static int freq = 1000;

static bool up = true;

// PIR
```

```
const int pirPin = 29;
// Ultrasonic
const int trigPin = 30;
const int echoPin = 31;
// Motor and buzzer
const int motorPin = 2;
const int buzzerPin = 43;
// LDR
const int ldrPin = A0;
// LEDs
const int yellowLeds[4] = {48, 49, 50, 51};
const int redLeds[4] = {44, 45, 46, 47};
// Motion counter
int motionCount = 0;
bool lastMotionState = LOW;
// 4-digit display timer
unsigned long motorRunStart = 0;
unsigned long totalMotorRunMs = 0;
bool lastMotorState = false;
void setup() {
  Wire.begin(8);
  Wire.onReceive(receiveEvent);
  lcd.begin(16, 2);
  dht.begin();
  pinMode(pirPin, INPUT);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(motorPin, OUTPUT);
  digitalWrite(motorPin, LOW);
  pinMode(buzzerPin, OUTPUT);
```

```
  noTone(buzzerPin);
  for (int i = 0; i < 4; i++) {
    pinMode(yellowLeds[i], OUTPUT);
    pinMode(redLeds[i], OUTPUT);
    digitalWrite(yellowLeds[i], LOW);
    digitalWrite(redLeds[i], LOW);
  }
  // 4-digit display setup
  byte numDigits = 4;
  byte digitPins[] = {10, 11, 12, 13};
  byte segmentPins[] = {4, 5, 6, 7, 8, 9, 38, 39};
  bool resistorsOnSegments = true;
  byte hardwareConfig = COMMON_CATHODE;
  bool updateWithDelays = false;
  bool leadingZeros = true;
  bool disableDecPoint = true;
  sevseg.begin(hardwareConfig, numDigits, digitPins, segmentPins,
               resistorsOnSegments, updateWithDelays, leadingZeros, disableDecPoint);
  sevseg.setBrightness(90);
  sevseg.setNumber(0, 0);
  Serial.begin(9600);
}
void loop() {
  int motion = digitalRead(pirPin);
  int lightValue = analogRead(ldrPin);
  bool nightMode = (lightValue < 800);
  // Motion counter
  if (motion == HIGH && lastMotionState == LOW) {
    motionCount++;
  }
  lastMotionState = motion;
```

```
  // Night mode / preferred temp
    activePreferredTemp = preferredTemp;


  float t = dht.readTemperature();
  float h = dht.readHumidity();
  float distance = getDistance();
  bool danger = (distance > 0 && distance < 20);
  bool motorOn = false;
  // Yellow LEDs = motion
for (int i = 0; i < 4; i++) {
  digitalWrite(yellowLeds[i], danger ? HIGH : LOW);
}
  // Red LEDs = night mode
  for (int i = 0; i < 4; i++) {
    digitalWrite(redLeds[i], nightMode ? HIGH : LOW);
  }
  // DHT error handling
  if (isnan(t) || isnan(h)) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("DHT Error");
    lcd.setCursor(0, 1);
    lcd.print("Check Sensor");
    digitalWrite(motorPin, LOW);
    noTone(buzzerPin);
    unsigned long errorStart = millis();
    while (millis() - errorStart < 1000) {
      sevseg.refreshDisplay();
    }
    return;
```

```
  }
  // Ultrasonic safety + motor logic
  if (danger) {
    digitalWrite(motorPin, LOW);
    motorOn = false;
    // COP SIREN only when object is close
    if (up) {
      freq += 30;
      if (freq >= 2000) up = false;
    } else {
      freq -= 30;
      if (freq <= 1000) up = true;
    }
    tone(buzzerPin, freq);
  }
  else {
    noTone(buzzerPin);
    if (distance >= 20) {
      digitalWrite(motorPin, HIGH);
      motorOn = true;
    } else {
      digitalWrite(motorPin, LOW);
      motorOn = false;
    }
  }
  // Motor run-time tracking
  if (motorOn && !lastMotorState) {
    motorRunStart = millis();
  }
  if (!motorOn && lastMotorState) {
    totalMotorRunMs += millis() - motorRunStart;
```

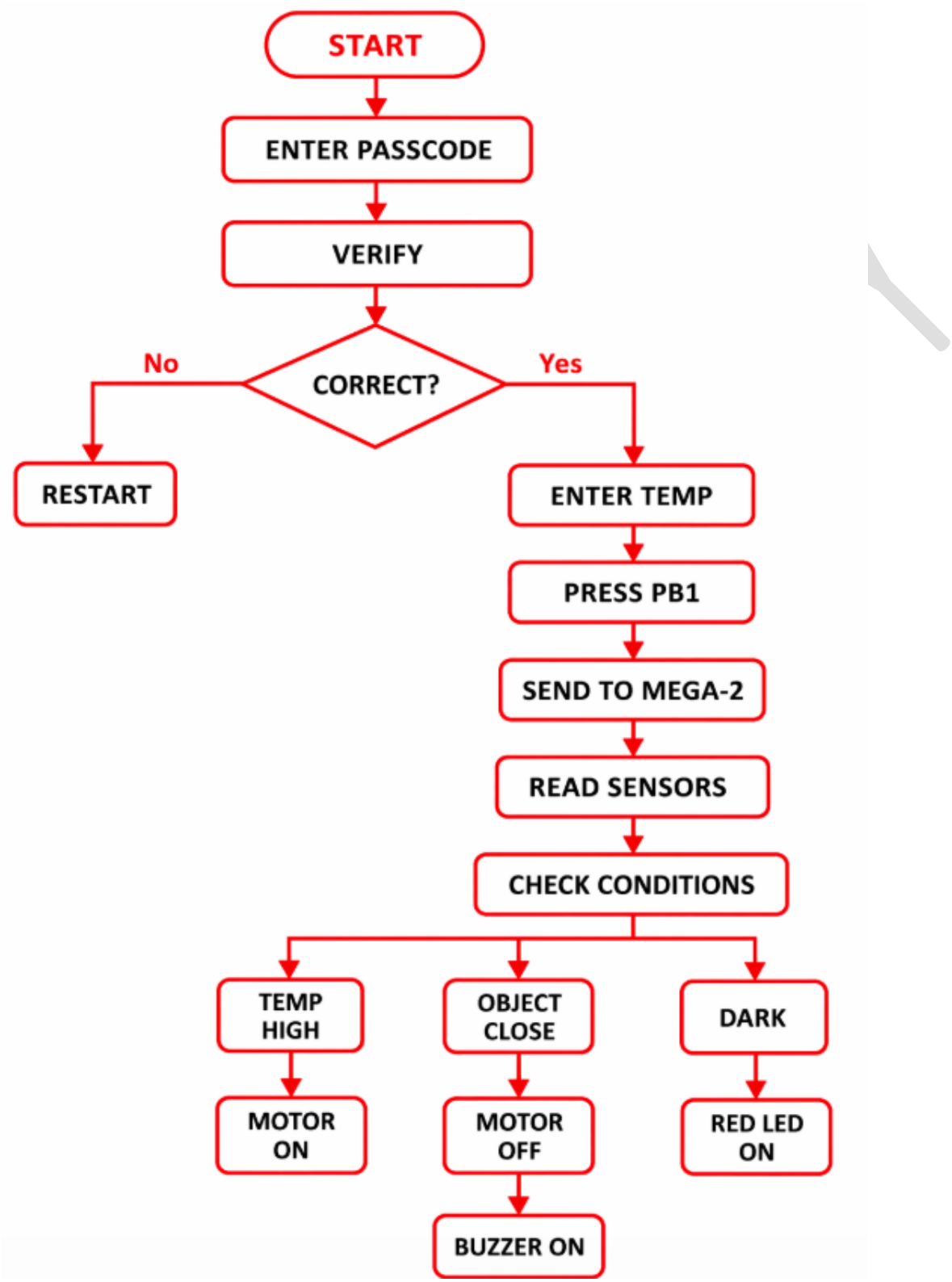
```
}
unsigned long currentRunMs = totalMotorRunMs;
if (motorOn) {
  currentRunMs += millis() - motorRunStart;
}
int runSeconds = currentRunMs / 1000;
if (runSeconds > 9999) runSeconds = 9999;
sevseg.setNumber(runSeconds, 0);
lastMotorState = motorOn;
// LCD display
lcd.clear();
if (nightMode) {
  lcd.setCursor(0, 0);
  lcd.print("Night Mode");
  lcd.setCursor(0, 1);
  lcd.print("SYS:");
  lcd.print(danger ? "STOP" : "RUN ");
  lcd.print(" C:");
  lcd.print(motionCount);
}
else if (danger) {
  lcd.setCursor(0, 0);
  lcd.print("SAFE STOP");
  lcd.setCursor(0, 1);
  lcd.print("D:");
  lcd.print(distance, 0);
  lcd.print("cm");
  lcd.print(" C:");
  lcd.print(motionCount);
}
```

```
else {
  lcd.setCursor(0, 0);
  lcd.print("T:");
  lcd.print(t, 1);
  lcd.print((char)223);
  lcd.print("C ");
  lcd.print("H:");
  lcd.print(h, 0);
  lcd.print("%");
  lcd.setCursor(0, 1);
  lcd.print("SYS:");
  lcd.print(motorOn ? "RUN " : "IDLE");
  lcd.print(" C:");
  lcd.print(motionCount);
}

// Serial output
Serial.print("Temp:");
Serial.print(t, 1);
Serial.print(",Hum:");
Serial.print(h, 1);
Serial.print(",Motor:");
Serial.print(motorOn ? "ON" : "OFF");
Serial.print(",Night:");
Serial.print(nightMode ? "ON" : "OFF");
Serial.print(",Motion:");
Serial.print(motion == HIGH ? "YES" : "NO");
Serial.print(",Count:");
Serial.print(motionCount);
Serial.print(",Dist:");
Serial.print(distance, 1);
```

```
  Serial.print(",Pref:");
  Serial.println(activePreferredTemp, 1);
  unsigned long refreshStart = millis();
  while (millis() - refreshStart < 500) {
    sevseg.refreshDisplay();
  }
}
void receiveEvent(int howMany) {
  if (Wire.available() >= 2) {
    byte highPart = Wire.read();
    byte lowPart = Wire.read();
    receivedTempRaw = word(highPart, lowPart);
    preferredTemp = receivedTempRaw / 10.0;
  }
}

float getDistance() {
  long duration;
  float distance;
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  duration = pulseIn(echoPin, HIGH, 30000);
  if (duration == 0) {
    return -1;
  }
  distance = duration * 0.034 / 2.0;
  return distance;
}
```

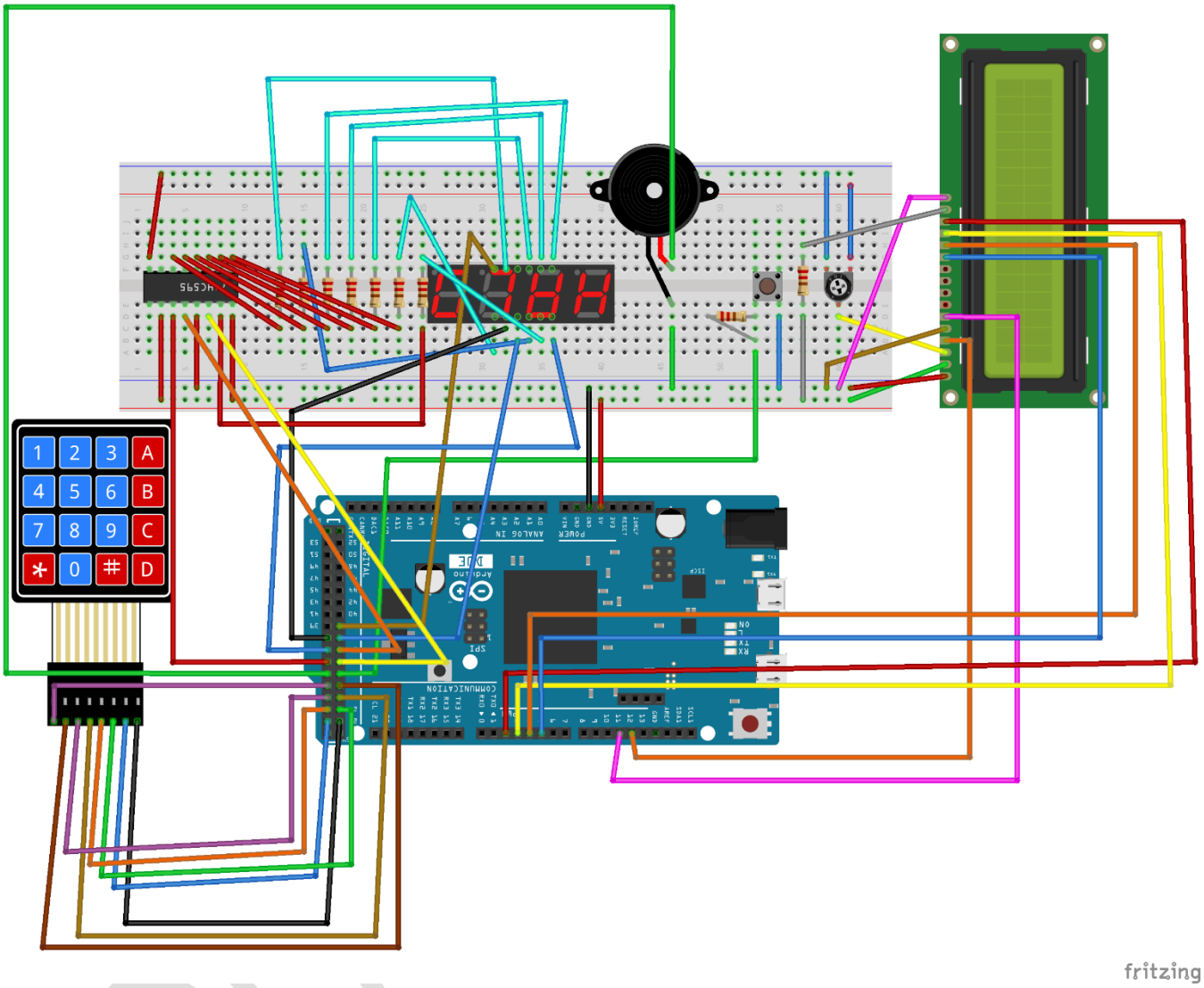
Appendix B

FlowChart

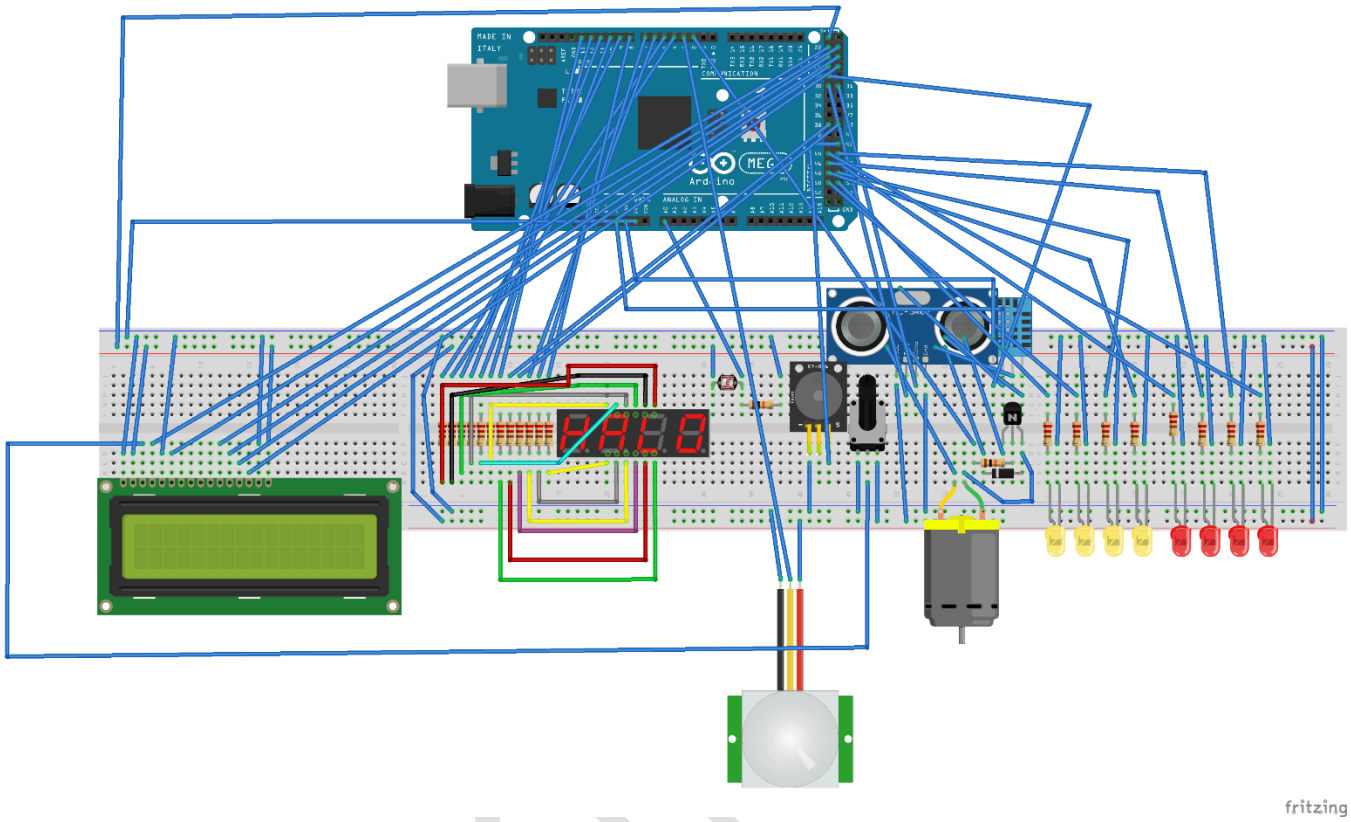


Fritzing Diagram

MEGA1

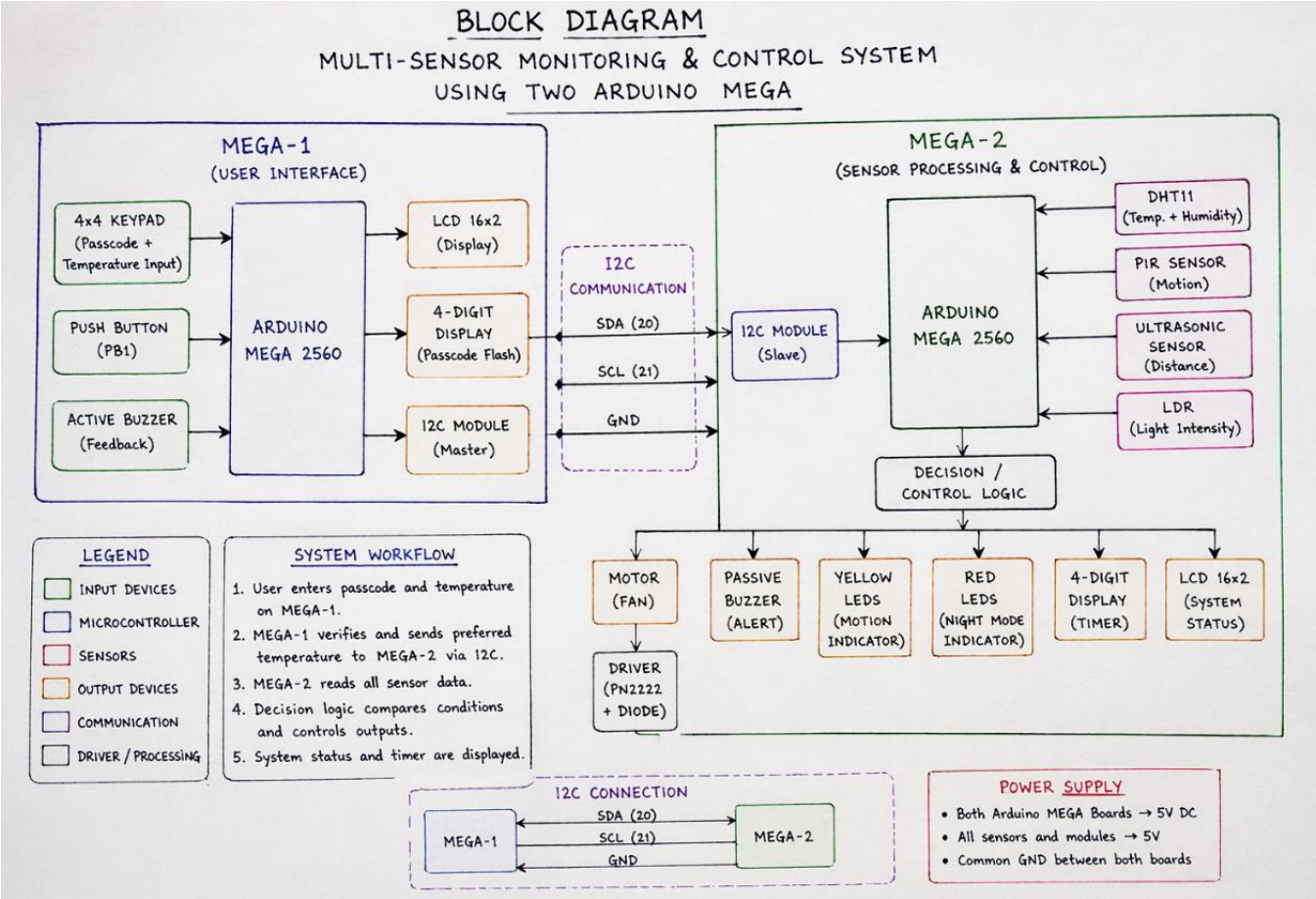


MEGA2



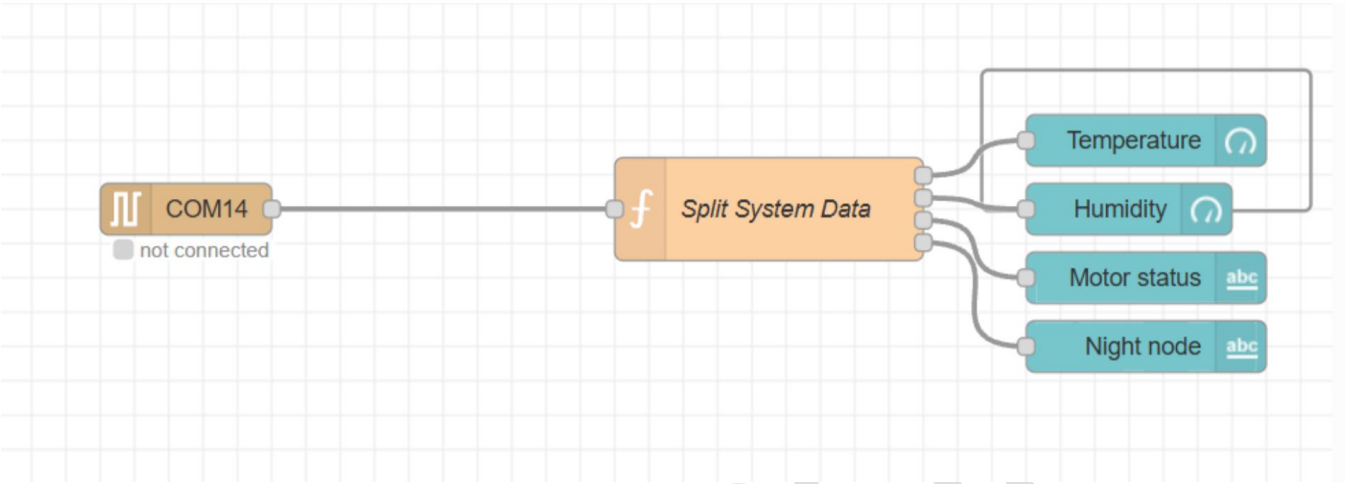
DO NOT

Block Diagram



This block diagram represents the overall system architecture showing the flow of data from user input to sensor processing and final outputs.

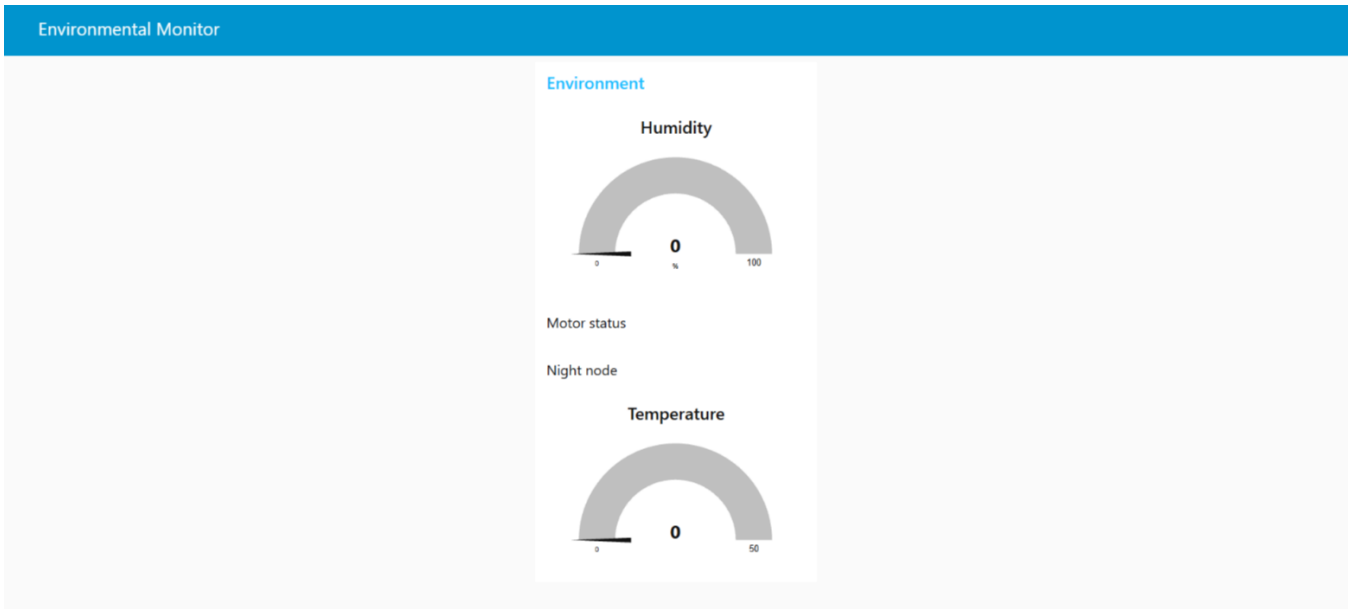
Node-RED flow diagram (data flow)



Node-RED was connected through serial communication to visualize real-time system data such as temperature, humidity, motor status, and system state.

DO NOT CO

Real-Time Monitoring Dashboard in Node-RED



The Node-RED dashboard was used to display real-time environmental data received from the Arduino system. The dashboard shows temperature, humidity, motor status, and night mode status in a visual and user-friendly format

DO NOT C